Energy Aware FPGA Implementation of Spiking Neural Network with LIF Neurons

Ali Oonwala, Arpit Arun Kumaar, Braden Schulz,
Grady Booth and Sreya Roy Chowdhury
University of Waterloo
Waterloo, ON, Canada
{aaoonwal, aakumaar, b2schulz, gjbooth, s4roycho}@uwaterloo.ca

I. Introduction: (Why is the project interesting?)

This work discusses the results of the 2024 paper Energy Aware FPGA Implementation of Spiking Neural Network with LIF Neurons [1]. The major breakthrough of this paper was that Spiking Neural Networks (SNNs) can perform ML inferencing with 86% higher energy efficiency than a competing Convolutional Neural Network. Leaky-Integrate-and-Fire, or LIF, neurons, work similarly within a Spiking Neural Network to how biological neurons work within the human brain: input values are accumulated over time, and can be positive or negative. The internally accumulated value has a slight exponential decay element. Once the internal value passes a threshold, the neuron spikes, and the internal value is reset to zero. Further spikes from the same neuron are suppressed for a cooldown period.

The particular problem that the paper focuses on is that of an automobile's dashcam, which must predict whether the vehicle is on a collision course based on a grayscale image. While a convolutional neural network would multiply the exact value of each pixel by a certain weight in the convolution, a spiking neural network translates the grayscale value of each pixel into a probability value of a spike; over several time-steps, the number of spikes accurately represents the value of each pixel. The beauty of this is that for each time-step, the model does not need to multiply the inputs by a weight; the weights can instead be multiplexed by the input, which will be either 0 or 1.

II. RTL Implementation Details:

The main accelerator components of our design are the lif module and cascaded adder. In the lif module, we set the spike output to be combinational, spiking on the same cycle as the internal value exceeds the threshold value. The reason we do this is so that we only need to interact with the lif for one cycle in a time-step; we are not providing data on one cycle and receiving data on the next. The purpose of the cascaded adder is to combine large numbers of weights that need to be added together. Each weight is paired with a weight on the same layer to produce a value that is one bit larger on the next layer, that must be subsequently paired, and so on until there is only one value left. This process has a latency of log2(N), N being the number of weights. The space requirement is linear with the number of weights.

To connect these modules, we have the main collision_predictor module. When designing this module, we faced some ambiguity in the paper's original design. Specifically, we were unsure whether the paper implemented a highly-parallelized design where many LIFs were calculated at the same time, or a more area-efficient design where a single LIF was used to sequentially calculate all of the second-layer spiking activity. We also considered a further reduction in area, where the cascaded adder was folded, calculating only 16, 64, or 128 additions within a cascaded system, then accumulating the results over several cycles. This was intended to reduce the power draw and LUT count for the design to a similar level to what was seen in the paper.

III. Testing Methodology: (How was testing and verification performed?)

Unit and integrations tests were created to verify our RTL implementations. All tests were implemented on the Cocotb Python test framework for SystemVerilog.

A. Unit Testing

We developed a suite of unit tests for both the cascaded adder module and lif-neuron module to ensure module-level functionality.

The cascaded adder unit tests covered the following: functional correctness, signed and boundary inputs, pipeline timing, reset behaviour, and robustness testing (feeding in garbage inputs while the input valid signal is de-asserted). The entire test suite was run over a 2-input, 512-input, and 4096-input cascaded adder, to ensure the module was correct under different parameter values of input size. A detailed list of every test case can be found at `cocotb/cascaded_adder/test_plan.md`.

The lif model was tested in a similar fashion, ensuring correctness in: reset behaviour, enable gating, integrate-leak-fire functionality, refractory period, and functional correctness with randomized inputs. A parameter sweep was done over the following configurations, ($k = 5, 2, 1$; $u_{th} = 100, 20, 8$; $u_{rest} = 0, 5, 0$; refactory_counter_max $= 5, 5, 2$), and we ensured tests were passing for all three. For functional correctness tests, our lif model output spikes were compared to a reference software model which was written in Python. A fully detailed test plan can be found at `cocotb/lif_model/test_plan.md`.

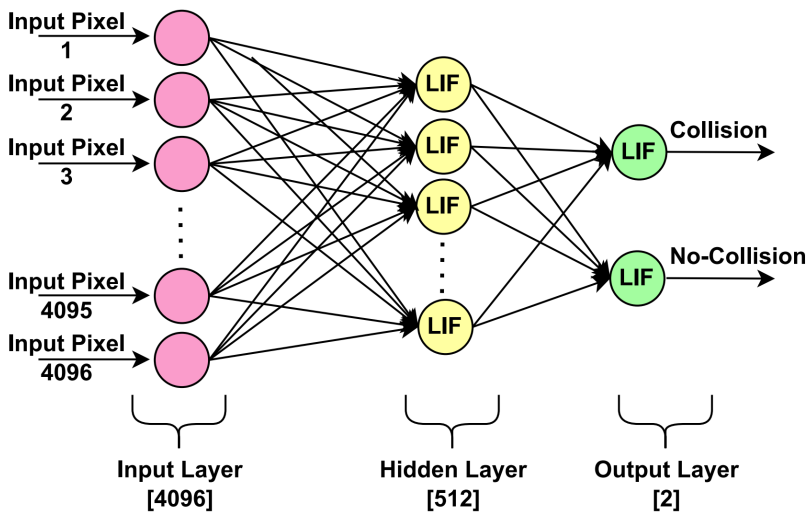

Fig. 1. Model architecture presented by Ali et al. [1] which our work aims to accelerate. The 25 time-steps are not shown in this image. Source: Ali et al. [1].

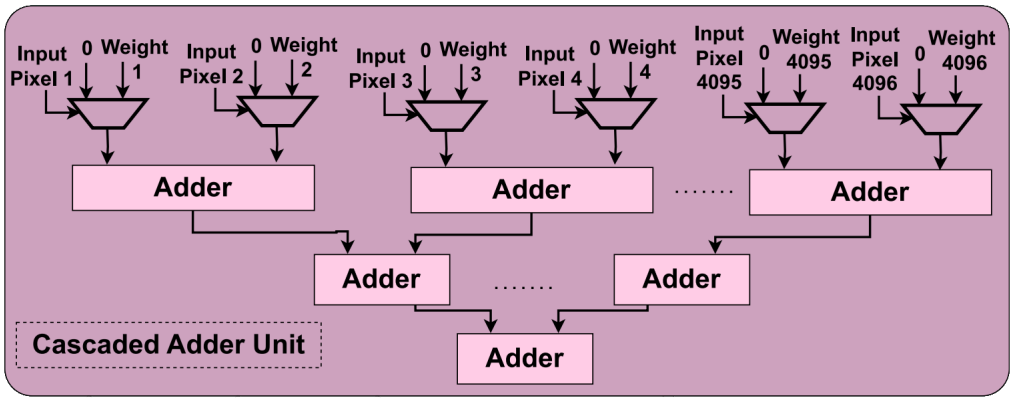

Fig. 2. Cascaded adder unit described by Ali et al. [1]. Source: Ali et al. [1].

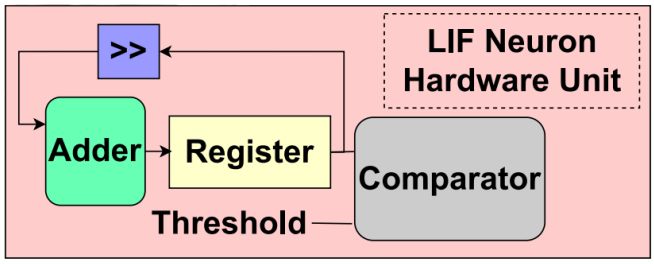

Fig. 3. LIF neuron module described by Ali et al. [1]. Source: Ali et al. [1].

B. Integration Testing

A complete test suite was created to validate our original two accelerator designs — the fully-parallel and resource-shared designs. The test inputs used were a set of 1576 images taken from the DroNet collision dataset, rescaled to 64x64 pixels, and converted to greyscale. The test suite contained the following tests.

Timing/latency test: 3 images are provided back to back and the latency of out_valid signals are verified.

Correctness test: All 1576 images are streamed and the outputs are verified against our reference software model. Accelerator's number of "collision" and "no-collision" spikes should match exactly with the software model instead of just the plain binary output.

Throughput test: For the fully-parallel design, images are streamed to fill up the pipeline 2 times over and out_valid signal frequency and amount are verified. For the resource-shared design, this test is a subset of the timing test.

All inclusive test: Latency, throughput, and correctness are verified in a single test, with back-to-back inputting to fill up the pipeline 2 times over.

As there was limited time and a relatively large search-space for accelerators, all other accelerators were only tested for correctness relative to the software model.

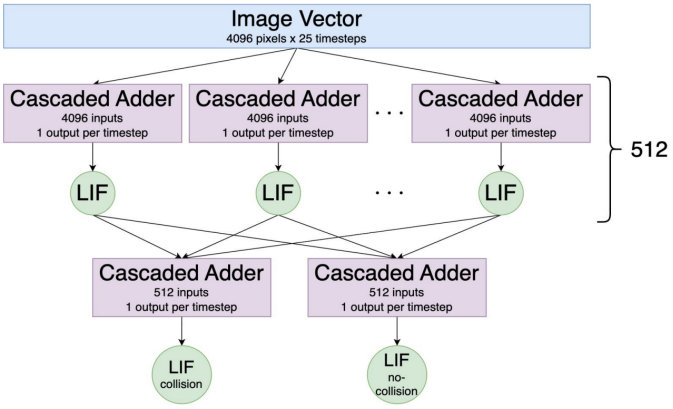

Fig. 4. Hardware architecture for the fully-parallel design (not synthesized).

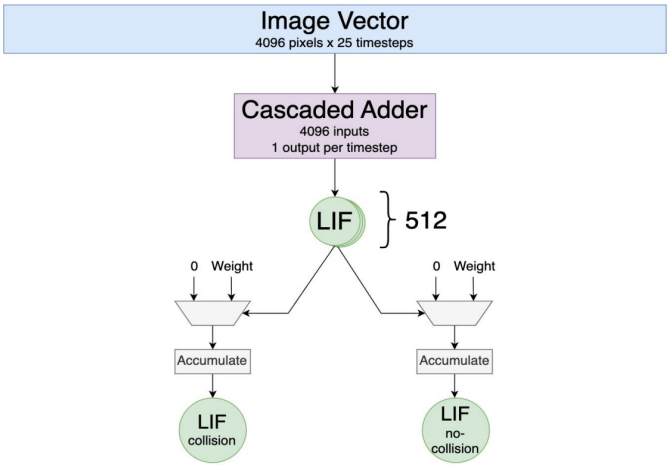

Fig. 5. Hardware architecture for the resource-shared design (not synthesized).

IV. Workloads & Software Simulator: (What were the workloads showed on the accelerator?)

A. Software Reference Model

The software reference model leveraged the snntorch Python package to train an SNN as described in the paper, using the same DroNet collision-avoidance dataset cited therein. The test set contained 1,576 images. The model converted each image to grayscale and resized it to 64 x 64 pixels. It encoded each image as a Bernoulli rate-coded spike train with 25 timesteps.

The network was a 4096–512–2 leaky integrate-and-fire (LIF) SNN with Q1.15 parameters. The software model achieved a train accuracy of 92.40% and a test accuracy of 82.4%. This compares favourably to the paper's stated train accuracy of 92% and test accuracy of 85%. [1]

RTL verification used deterministic spike traces generated by the software reference model and replayed using Verilator. The sequential testbench is configured to check the first three images, while the folded designs check smaller subsets. The 128-wide folded SNN accelerator design particularly checks 2 images under normal operation and one image with weight-stream backpressure.

RTL verification used deterministic spike traces generated by the software reference model and replayed using Verilator. The sequential testbench is configured to check the first 3 images, while the folded designs check smaller subsets. The 128-input design checks two images under normal operation and one image with weight-stream backpressure. Consequently, these tests establish bit-exact agreement only for the evaluated subset, not full-test-set RTL accuracy.

B. Workloads

1) Dataset and Verification Workload: The software reference model was evaluated on the full 1,576 image DroNet test set. RTL functional verification replayed a deterministic subset of the software-generated spike traces. Post-route power analysis used switching activity (SAIF) from one complete trained-image inference.

V. RTL Implementation

We implement the 4096-512-2 LIF SNN using 3 accelerator designs. Every design uses the same Q1.15 weights, biases, and LIF rule. They differ only in how much each 4096-wide inner product is laid out in space, and in whether the network is simulated in Verilator or placed on an Artix UltraScale+ AU25P.

A. Cascaded Adder & LIF Neuron Hardware Unit

Following the paper, a single neuron is a cascaded adder followed by a LIF neuron hardware unit. Because input spikes are binary, the cascaded adder replaces a multiply-accumulate: each spike selects its 16-bit weight or zero, and a pipelined adder tree reduces those terms. Bias is added after the tree. The LIF unit then updates membrane potential, applies decay and a 5 step refractory period, and emits a binary spike when the potential exceeds the learned threshold. These two blocks are the arithmetic core of every design below.

B. Parallel and Resource-Shared Designs

We first built two full-network RTL models in order to verify the 4096-512-2 mapping independently of FPGA area limits. Weights and biases are preloaded from the software-exported hexadecimal tables. Input spike trains are encoded offline and replayed in simulation. Unfortunately, neither design is placed on the FPGA due to resource/area limitations on available hardware.

The parallel design instantiates 512 hidden-layer cascaded adders and 512 LIF units, then two 512-input cascaded adders and two output LIF units (collision and no-collision). All hidden neurons evaluate a timestep together. This is the throughput-oriented unrolling of the network and an upper bound on full-network parallelism. A 512-wide bank of 4096-input trees, plus the on-chip layer-1 matrix, does not fit the AU25P.

The resource-shared design instantiates one 4096-input cascaded adder and reuses it across the 512 hidden neurons, so each timestep takes 512 neuron cycles. Hidden spikes

are folded into the two output classes with a spike-gated accumulate, rather than two 512-input trees. This is the area-oriented full-network golden model. It still does not fit the device: the adder remains a full 4096-input tree, and the layer-1 weights remain on chip.

These two designs are the functional references. They only establish bit-level behavior and a throughput-versus-area bound and are not the source of LUT, FF, BRAM, frequency, or power results.

C. Folded FPGA Designs

This work is an extension of our paper presentation. Placing a full 4096-input cascaded adder, let alone 512 of them, is not feasible on the AU25P, and the layer-1 matrix is $\sim$4 MiB. The idea is to time-multiplex each 4096-wide weighted sum onto a narrower datapath that evaluates P synapses per cycle. The synthesized networks fold each 4096-wide inner product onto a narrower datapath and stream layer-1 weights. Each cycle evaluates P synapse-weight products in parallel. We call each of those products a lane. One copy of the P-wide datapath is an engine. Completing one hidden neuron then takes $4096/P$ cycles of partial products, which we accumulate before the LIF update. Input spikes for all 25 timesteps are buffered on chip; only one weight row is resident at a time, with two row buffers so that neuron $(n+1)$ can load while neuron n computes.

We placed-and-routed 4 folded accelerators:

- a single-engine, 16-lane design ($P = 16$)
- a single-engine, 64-lane design ($P = 64$)
- a single-engine, 128-lane design ($P = 128$)
- a 4x engine, 16-lane design, which uses four 16-lane engines on the same spike word so that four hidden neurons compute at once

The 16 and 64-lane engines, as well as the 4 engine design, accept a 64-bit weight stream (rather than storing weights on chip). The 128-lane design accepts a 128-bit stream and reduces each fold with eight pipelined 16-input cascaded adders. Folding here is not the same as the resource-shared design which has a 4096-input tree. The folded designs narrow that tree to P lanes so that a full inference fits the FPGA.

To compare a paper-style datapath without placing the full network, we also implement one cascaded adder and LIF neuron hardware unit out of context: a single 4096-input tree plus one LIF. That partition measures the paper's per-neuron arithmetic. It is not a 4096-512-2 FPGA mapping.

LUT, FF, BRAM, frequency, and power estimates in later sections come only from the folded accelerators and from this out-of-context unit. Cycle-accurate full-network checks of the unreduced 4096-input mapping to use the parallel and resource-shared designs.

VI. (Comparison between Software Simulator and RTL Simulation Results, Comparison between Obtained and Published Results, Discussion/Commentary on the Results)

We report inferences per second for each routed design. This metric counts "completed work". It includes stalls, weight streaming, control overhead, and the fold factor. A typical GOPS value does not include these effects, because it is only the product of the lanes in the datapath and the clock rate. The operation count is also not standard for an SNN. A binary spike selects a weight and then adds it. Some authors count this as one operation. Other authors count it as two, because they compare it to a multiply-accumulate. This choice changes the reported GOPS by a factor of two. One inference is the same unit of work for an SNN, a BCNN, and a CNN. It therefore permits a direct comparison. It also gives energy per inference: our 128-lane design does 601.35 inferences/s at 1.165 W, which is 1.94 mJ per inference. We cannot calculate this value for [1], because that work gives no inference rate.

Our inference of the published 541 GOPS is that it's for depicting peak arithmetic rate, not an end-to-end throughput. A 4096-input cascaded adder does 4096 select-and-add operations in each cycle. At 67 MHz and two operations for each synapse, this gives 548.9 GOPS, which agrees to within 1.5%. The value thus applies to one adder unit, and not to the network. The full network needs 52.45 M synapse evaluations for each inference across the 25 timesteps, which is 104.9 Mop. At 541 GOPS, the datapath must do 4,036 synapses in each cycle and read 65,536 weight bits in each cycle. This is approximately 549 GB/s, which no Artix-7 memory system supplies.

There are two other items from their results which are absent or inconsistent. First, the layer-1 weight matrix is 4096 x 512 x 16 bits, which is 4 MiB. This is more than the BRAM of an Artix-7. The paper gives no BRAM count and no external memory interface, so the weight delivery path is unknown. This path sets the true inference rate. Second, the reported 6,521 LUTs cannot hold one 4096-input adder tree of 16-bit terms. The first level of that tree alone needs 2,048 adders of 17 bits. The area value thus describes a folded datapath, but the GOPS value describes a full-width datapath. The two values cannot come from the same netlist.

VII. Conclusion & Future Work

In this paper we sought to reproduce past work's [1] findings that SNNs may perform inference with superior energy efficiency when compared to a contemporary CNN baseline. Our research surfaced many questions that appeared unanswered by the original paper, such as the size of cascaded adder used and number of LIF neuron modules instantiated to achieve their low utilisation numbers. These ambiguities were addressed by our sweep of the design space, wherein we produced RTL which wrapped the key cascaded adder and LIF neuron modules with different sets of control logic and varied parametrization to produce end-to-end numbers for implementation, post place-and-route, and SAIF power analysis for a multitude of designs. Through this method we were able to discover designs which rival the paper's utilization, but none of these closely match its claimed power consumption.

Future research in this regard could work to close the power gap we observed more precisely by tuning the frequency of

TABLE I
RESULT

Result	Paper-Reported SNN	BCNN baseline	Our design @67 MHz	Our design @280 MHz
Hardware scope	4096-512-2 SNN	VGG-11 BCNN	One 4096-input tree + one LIF	One 4096-input tree + one LIF
FPGA	Artix-7	Zynq-7000	AU25P UltraScale+	AU25P UltraScale+
Frequency	67 MHz	143 MHz	67 MHz	280 MHz
Power	0.495 W	2.3 W	1.332 W total*, 0.875 W dynamic	3.874 W total*, 3.395 W dynamic
Peak GOPS	541	329	548.88 (peak synaptic arith rate)	2294
Peak GOPS/W	1093	143	412	592
LUTs	6521	Not given	38667	38713
Registers/FFs	1780	Not given	69606	69606

TABLE II
COMPARISON BETWEEN OBTAINED AND PUBLISHED RESULTS

Design	Clock	Throughput	Inference/s	LUT / FF	BRAM	Power	GOPS/W
Paper, as published	67 MHz	541 GOPS reported	Not reported	6,521 / 1,780	Not reported	0.495 W	1,093
Our 64-lane routed core	125 MHz	15.68 effective GOPS	149.49	5,130 / 3,348	34.5	0.495 W	31.68
Our 256-lane routed core	67 MHz	13.36 effective GOPS	127.34	23,061 / 6,559	119.5	0.530 W	25.21

TABLE III
POST-ROUTE FOLDED ACCELERATOR RESULTS

Design	Engines × lanes	Weight bus	Passing freq.	Cycles/inference	Inferences/s	Effective GOPS	LUT	FF	BRAM tiles	Power	GOPS/W
E4/L16	4×16	64-bit	160 MHz	836,514	191.27	20.07	5,017	5,364	21.5	0.621 W	32.31
L16	1×16	64-bit	175 MHz	3,298,588	53.06	5.57	2,285	2,530	13.5	0.509 W	10.94
L64	1×64	64-bit	125 MHz	836,188	149.49	15.68	5,130	3,348	34.5	0.581 W	26.99
L128	1×128	128-bit	250 MHz	415,731	601.35	63.09	11,482	6,435	64.5	1.165 W	54.15

these designs along with further varying the number of adder blocks in use. Additionally, our low-area design results were based on calculating the results for one time-step at a time, which means that we required a module which carried the internal value of every LIF from one time step to the next. To avoid this, we could calculate the spikes from one LIF at a time for all time-steps. This was one of the changes we attempted to implement in our design with a folded cascaded adder, but we have yet to see whether this has a power-reduction effect in the larger two designs.

Meanwhile, we have demonstrated that our parallel architecture produces correct results in simulation, though we were ultimately unable to acquire utilisation numbers for this design as its immense resource demands would have required access to higher-end architectures with adequate resources for its implementation. Future work may implement and run power analysis on this design to assess the results it would produce on a large chip. If its GOPS/W far exceed that of the folded design explored in this paper, this larger design's use may be justified in alternative applications to edge device inference. Due to the expected increase in static power draw for a large design such as this, it would be expected to only intermittently keep the chip powered. With this in mind, we expect that this design should power on to complete a large workload with more efficient GOPS/W prior to shutting off to conserve power and accumulate work in advance of its next run.

VIII. LINK TO REPOSITORY:

https://git.uwaterloo.ca/ece493-s26/projects/team-22

REFERENCES

[1] Asmer Hamid Ali, Mozhgan Navardi, and Tinoosh Mohsenin, "Energy-Aware FPGA Implementation of Spiking Neural Network with LIF Neurons," arXiv:2411.01628, 2024.